Case_in_HPC_Assign1

Yunyi Xiang

February 2026

Git Repository

https://github.com/xiangyu116/Case-Studies-in-HPC/tree/main/Assign1

1 Implement TSQR (4 blocks) in Matlab and verify its correctness

First, follow the methods described in the materials presented in class. Divide A evenly into 4 sub-blocks in the row direction (requiring that $4 \mid m$):

$$A = \begin{bmatrix} A_1 \\ A_2 \\ A_3 \\ A_4 \end{bmatrix}, \qquad A_i \in \mathbb{R}^{\frac{m}{4} \times n}.$$

Step 1: Local QR for Each Sub-block Perform the reduced QR decomposition on each block (in Matlab, use `qr(Ai,0)`):

$$A_i = Q_i R_i, \qquad Q_i \in \mathbb{R}^{\frac{m}{4} \times n}, \;\; R_i \in \mathbb{R}^{n \times n}, \quad i = 1, 2, 3, 4.$$

Step 2: Perform Tree Reduction on R_i Reduce the small-scale matrix R_i through two layers of tree structure reduction:

$$\begin{bmatrix} R_1 \\ R_2 \end{bmatrix} = Q_{12} R_{12},$$
$$\begin{bmatrix} R_3 \\ R_4 \end{bmatrix} = Q_{34} R_{34},$$
$$\begin{bmatrix} R_{12} \\ R_{34} \end{bmatrix} = Q_{1234} R,$$

Step 3: Construct the Global Q The code combines the local Q with the tree-based reduced Q using a block diagonal matrix (`blkdiag`):

$$Q = \text{blkdiag}(Q_1, Q_2, Q_3, Q_4) \cdot \text{blkdiag}(Q_{12}, Q_{34}) \cdot Q_{1234},$$

Thus, we obtain $Q \in \mathbb{R}^{m \times n}$, and it satisfies the approximate relationship

$$A \approx QR.$$

1.1 Verification Indicators of Correctness

According to the code, the Frobenius norm is used to calculate two key errors:

(1) Relative decomposition error

$$\text{rel_err} = \frac{\|A - QR\|_F}{\|A\|_F}.$$

(2) Orthogonality Error

$$\text{orth_err} = \|Q^T Q - I\|_F.$$

1.2 Experimental Results

The output of running the program is:

$$\text{rel_err} = 3.415 \times 10^{-16}, \qquad \text{orth_err} = 2.309 \times 10^{-15}.$$

```
>> Assign1
Relative factorization error ||A-QR||_F / ||A||_F = 3.415e-16
Orthogonality error ||Q^TQ-I||_F = 2.309e-15
```

Figure 1: Experimental Results

1.3 Conclusion

- The decomposition error $\|A - QR\|_F / \|A\|_F$ is extremely small, indicating that the calculated Q and R can highly reconstruct the original matrix A;
- The orthogonality error $\|Q^T Q - I\|_F$ is also very small, suggesting that the constructed Q is approximately orthogonal;
- Therefore, the 4-block TSQR method implemented in this problem is consistent with the communication-avoiding TSQR method described in class, and it is implemented correctly and numerically stable.

2 Implementing TSQR using MPI and verifying its correctness

Algorithm Idea (4 Processes, Tree Reduction)

Let $A \in \mathbb{R}^{m\times n}$ ($m \gg n$). Divide A evenly along its rows into 4 blocks:

$$A = \begin{bmatrix} A_0 \\ A_1 \\ A_2 \\ A_3 \end{bmatrix}, \qquad A_r \in \mathbb{R}^{\frac{m}{4}\times n}, \text{ where } r = 0, 1, 2, 3.$$

Each MPI process r only holds the local block A_r, and performs the reduced QR decomposition locally first:

$$A_r = Q_r R_r, \qquad Q_r \in \mathbb{R}^{\frac{m}{4}\times n},\ R_r \in \mathbb{R}^{n\times n}.$$

Then, the small matrix R_r undergoes tree-based reduction (the key to communication avoidance: only communicating the $n \times n$ matrix R):

$$\begin{bmatrix} R_0 \\ R_1 \end{bmatrix} = Q_{01} R_{01},$$
$$\begin{bmatrix} R_2 \\ R_3 \end{bmatrix} = Q_{23} R_{23},$$
$$\begin{bmatrix} R_{01} \\ R_{23} \end{bmatrix} = QR,$$

Finally, the global upper triangular matrix R is obtained at rank 0, satisfying $A = QR$ (where Q can be constructed by back-substitution based on the tree structure; the key point of this implementation is R and communication avoidance).

2.1 Code Implementation

- Rank 0 generates a random matrix A (stored in column-major order) and packs it row-wise before distributing it to 4 processes using `MPI_Scatter`;
- Each process calls LAPACK's `dgeqrf` on its local block to perform QR decomposition and extract R_r;
- Process 1 sends R_1 to Process 0; Process 3 sends R_3 to Process 2;
- Process 2 computes $R_{23} = \mathrm{qr}([R_2; R_3])$ and sends it to Process 0;
- Process 0 computes $R_{01} = \mathrm{qr}([R_0; R_1])$, and then calculates the final $R = \mathrm{qr}([R_{01}; R_{23}])$.

2.2 Correctness Verification: Verifying $A^T A \approx R^T R$

Since $A = QR$ and Q is orthogonal (i.e., $Q^T Q = I$), theoretically, it should be the case that

$$A^T A = R^T Q^T Q R = R^T R.$$

Therefore, the relative error is calculated on rank 0.

$$\text{err} = \frac{\|A^T A - R^T R\|_F}{\|A^T A\|_F}.$$

Output:

$$\text{err} = 6.168 \times 10^{-16}.$$

This error magnitude is close to the machine precision of double-precision floating-point (approximately 10^{-16}), indicating that the obtained R is highly consistent with the original matrix A in terms of numerical values, and the TSQR implementation is correct.

3 Performance Testing of TSQR

3.1 Timing Method and Output Format

To investigate the trend of the running time of TSQR as the matrix size changes, I used `MPI_Wtime()` in the code for timing. Before and after calling TSQR, I set barriers for synchronization (`MPI_Barrier`) to avoid inconsistent timing intervals among different processes:

$$t = t_1 - t_0.$$

Finally, only one result is output by rank 0:

$$m \quad n \quad t,$$

And redirect and save it as a text file (for example, `result_m.txt`, `result_n.txt`), which will be used for plotting.

3.2 Experiment 1: Fixed $n = 50$, varying m ($p = 4$)

Keep the number of columns $n = 50$ constant, gradually increase the number of rows m, record the running time and plot the curve (Figure 2). This experiment is used to observe the scalability of TSQR in the tall-skinny scenario as m expands.

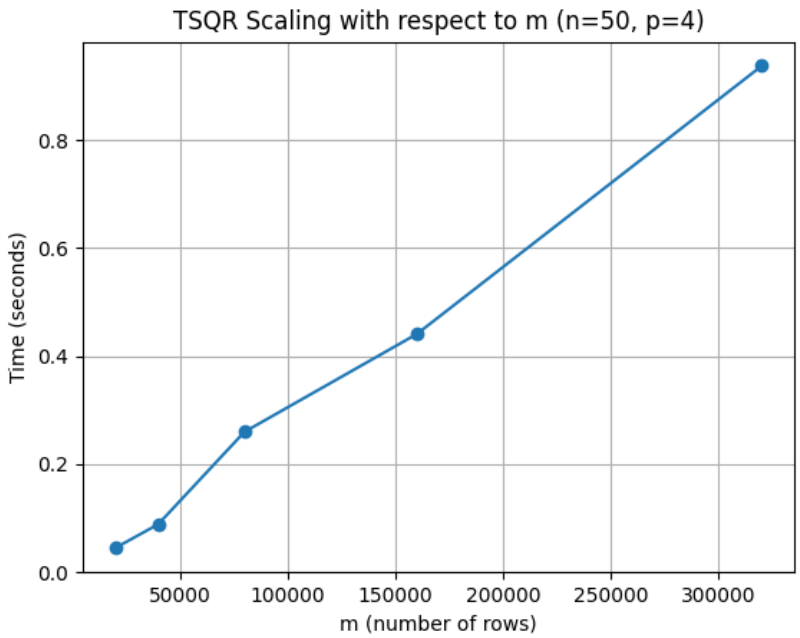

Figure 2: TSQR scaling with respect to m (with $n = 50$ and $p = 4$)

As can be seen from the figure: as m increases, the total time shows an overall upward trend and approximately linear growth. This is reasonable because the main computational load of the local QR is approximately proportional to m (the size of the local block is $m/4$).

3.3 Experiment 2: Fixed $m = 160000$, varying n ($p = 4$)

Keep the number of rows $m = 160000$ unchanged, gradually increase the number of columns n, record the running time and plot the curve (Figure 3). Also plot a log-log graph (Figure 4) to observe the growth trend.

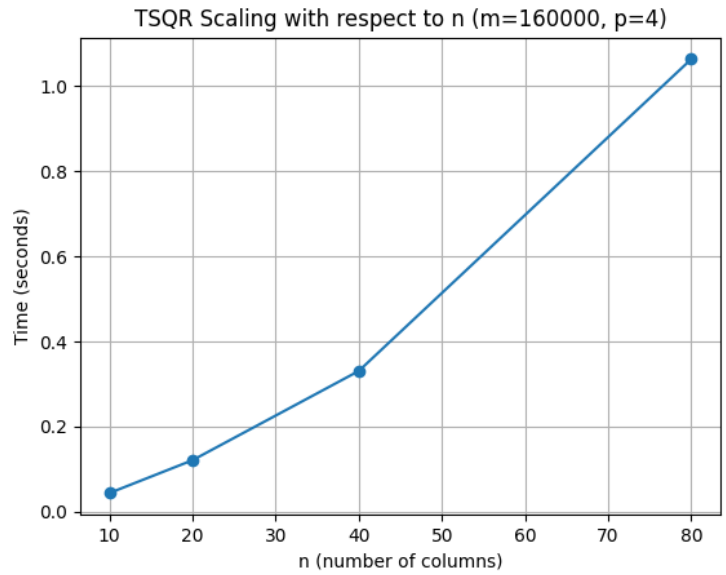

Figure 3: TSQR scaling with respect to n (with $m = 160000$ and $p = 4$)

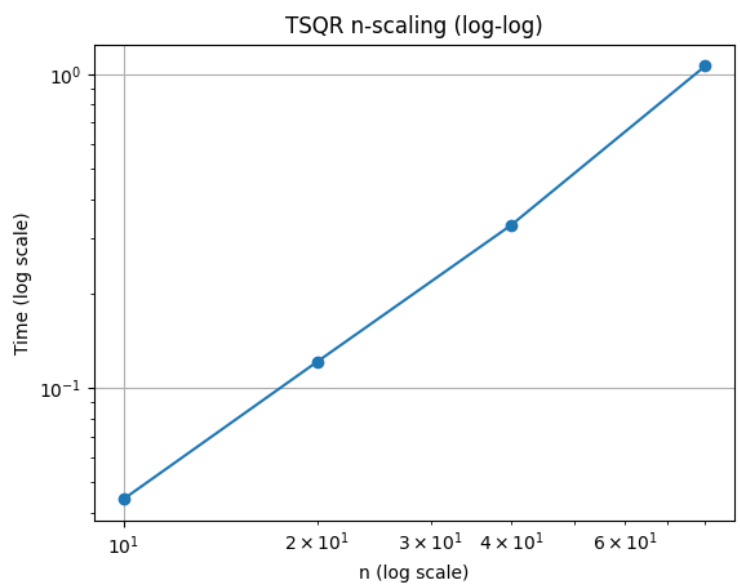

Figure 4: TSQR scaling with respect to n (in log-log perspective)

From the results, it can be seen that: as n increases, the running time rises more significantly (more "steeply" than when m increases). This is also reasonable: for TSQR, the computational complexity of the local QR calculation is more closely related to n (the typical dominant term is $\mathcal{O}(mn^2)$), and the tree reduction stage also involves performing QR on small matrices (related to the power growth of n), so when n increases, the total time will rise even faster.

3.4 Conclusion

Based on the results of the two sets of scaling experiments, it can be concluded that:

- When n is fixed, the running time of TSQR increases approximately linearly with the increase of m;
- When m is fixed, the running time of TSQR increases even faster as n increases, indicating that the number of columns has a more significant impact on the computational load;
- These trends are consistent with the computational structure of TSQR: local QR processing is performed on $m/4 \times n$ sub-blocks, and communication only occurs to transfer small $n \times n$ matrices during the reduction stage.